

Implementation of `sigalarm` and `sigreturn` in xv6

SRIKARTHIK KANITHI

Roll No: CS23B029

1 Overview

This report documents the changes made to the xv6 operating system to implement user-level alarm handling using two new system calls: `sigalarm()` and `sigreturn()`. Each modification is shown along with the file and the exact place where the code was added. A brief explanation and the execution flow are also included.

2 Changes Made

2.1 Makefile

Added the alarm test program to the UPROGS section:

```
UPROGS += $U/_alarmtest
```

Code 1: Code: Makefile modification

2.2 user.h

Added syscall declarations in the system calls section:

```
int sigalarm(int, void (*handler)());  
int sigreturn(void);
```

Code 2: Code: user.h - declarations

2.3 usys.pl

Added syscall entries in the entries section:

```
entry("sigalarm");  
entry("sigreturn");
```

Code 3: Code: usys.pl - syscall entries

2.4 trap.c

Added inside the `usertrap()` function, within the `if(which_dev == 2)` block (timer interrupts):

```

if(p && p->state == RUNNING && p->alarm_interval > 0){
    if(!p->handling_alarm){
        p->alarm_elapsed++;
        if(p->alarm_elapsed == p->alarm_interval){
            *p->alarm_trapframe = *p->trapframe;
            p->trapframe->epc = (uint64)p->alarm_handler;
            p->handling_alarm = 1;
            p->alarm_elapsed = 0;
        }
    }
}
}

```

Code 4: Code: trap.c - alarm handling in usertrap()

2.5 syscall.h

Added syscall numbers:

```

#define SYS_sigalarm 24
#define SYS_sigreturn 25

```

Code 5: Code: syscall.h - new syscall numbers

2.6 syscall.c

Added in two places:

- In the syscall dispatch table.
- In the prototypes section.

```

[SYS_sigalarm] sys_sigalarm,
[SYS_sigreturn] sys_sigreturn,

extern uint64 sys_sigalarm(void);
extern uint64 sys_sigreturn(void);

```

Code 6: Code: syscall.c - dispatch table entries / prototypes

2.7 proc.h

Added new fields to struct proc:

```

int alarm_interval;
void (*alarm_handler)();
int alarm_elapsed;
int handling_alarm;
struct trapframe *alarm_trapframe;

```

Code 7: Code: proc.h - new fields in struct proc

2.8 proc.c

Added in allocproc() after allocating the main trapframe:

```
p->alarm_handler = 0;
p->alarm_interval = 0;
p->alarm_elapsed = 0;
p->handling_alarm = 0;

p->alarm_trapframe = (struct trapframe*)kalloc();
if(!p->alarm_trapframe){
    freeproc(p);
    release(&p->lock);
    return 0;
}

memset(p->alarm_trapframe, 0, sizeof(struct trapframe));
```

Code 8: Code: proc.c - allocproc additions

Added in freeproc() after deallocating the main trapframe:

```
if(p->alarm_trapframe){
    kfree((void*)p->alarm_trapframe);
    p->alarm_trapframe = 0;
}
```

Code 9: Code: proc.c - freeproc alarm cleanup

2.9 sysproc.c

Added at the end of the file:

```
uint64
sys_sigalarm(void)
{
    int n;
    uint64 fn;
    argint(0, &n);
    argaddr(1, &fn);
    struct proc *p = myproc();
    if(n <= 0 ){
        p->alarm_interval = 0;
        p->alarm_handler = 0;
        p->alarm_elapsed = 0;
        p->handling_alarm = 0;
        return 0;
    }
    p->alarm_interval = n;
    p->alarm_handler = (void(*)())fn;
    p->alarm_elapsed = 0;
    p->handling_alarm = 0;
    return 0;
}
```

Code 10: Code: sysproc.c - *sys_sigalarm*

```
uint64
sys_sigreturn(void)
{
    struct proc *p = myproc();
    if(!p->alarm_trapframe)
        return -1;
    *p->trapframe = *p->alarm_trapframe;
    p->handling_alarm = 0;
    return p->trapframe->a0;
}
```

Code 11: Code: sysproc.c - *sys_sigreturn*

3 Brief Explanation

- `sigalarm()` stores the clock interval and the handler function address.
- Between two `sigalarm` calls, the process keeps these values.
- `sigalarm(0,0)` acts as a base case to reset interval and disable alarms.
- On every timer interrupt, the kernel increments a counter.
- When the counter equals the interval:
 - The current trapframe is saved into a separate structure.
 - `epc` is updated to point to the handler function.
 - The alarm state is updated to avoid reentrant alarms.
- When the kernel returns to user mode, execution jumps to the handler.
- The handler must call `sigreturn()` at the end.
- `sigreturn` restores the original trapframe (including PC and registers).
- Execution then continues at the instruction where the process was interrupted.

4 Execution Flow

1. User calls `sigalarm(n, handler)` → kernel stores values in proc struct.
2. Timer interrupts trigger `usertrap()`, with `which_dev == 2`.
3. Alarm counter increments; when equal to interval:
 - Save trapframe into `alarm_trapframe`.
 - Redirect `epc` to the handler.

- Set `handling_alarm = 1` and reset counter.
4. Returning to user mode starts execution at the handler.
 5. Handler eventually calls `sigreturn()`.
 6. Kernel restores the saved trapframe and clears `handling_alarm`.
 7. Process resumes execution at the point it was interrupted.